

Task Automation Tool

Submitted By

Ardra Rajesh

Internship

CODEVEDX Python Programming Internship

Department

B.Tech Computer Science & Engineering

Date

06 July 2026

Table of Contents

1. Introduction
2. Objectives
3. Features
4. Technologies Used
5. Python Modules Used
6. Project Workflow
7. Project Structure
8. Screenshots
9. Sample Output
10. Error Handling
11. Future Enhancements
12. Conclusion

Introduction

Automation has become an essential part of modern computing, helping users reduce repetitive manual work and improve productivity. Tasks such as organizing files, generating reports, and sending notifications can consume significant time when performed manually. Automating these processes improves efficiency, minimizes human error, and saves valuable time.

The **Task Automation Tool** is a Python-based console application developed to automate common file management tasks. It automatically organizes files into categorized folders based on their file extensions, generates detailed reports of completed operations, and optionally sends email notifications with task summaries.

Unlike a basic automation script, this project supports both an interactive menu and command-line execution, making it flexible for different users. It also includes logging, dry-run mode, duplicate file handling, secure email configuration using environment variables, and robust exception handling. Since it is built entirely using Python's standard library, no external packages are required.

This project helped me understand practical automation techniques, file and directory management, report generation, logging, email integration, command-line programming, and secure software development using Python.

Objectives

The main objectives of this project are:

- To automate repetitive file management tasks using Python.
- To organize files automatically based on their extensions.
- To generate detailed automation reports.
- To send email notifications after task completion.
- To maintain activity logs for monitoring and debugging.
- To implement secure email authentication using environment variables.
- To improve programming skills using file handling, logging, and command-line programming.
- To understand real-world automation using Python.

Features

File Organization

- Automatically organize files into categorized folders.
- Categorize Images, Documents, Videos, Audio, Archives, Scripts, and Others.
- Automatically create folders when required.
- Prevent overwriting by renaming duplicate files.
- Skip hidden files and system files.
- Ignore directories while processing files.

Dry Run Mode

- Preview all file movements without making actual changes.
- Verify automation before execution.
- Safe testing environment.

Report Generation

- Generate detailed automation reports.
- Export reports as Text (.txt).
- Export reports as CSV (.csv).
- Include processed, moved, skipped, and failed files.
- Store reports for future reference.

Email Notifications

- Send task completion summaries through email.
- Attach generated reports automatically.
- Use Gmail App Password authentication.
- Keep passwords secure using environment variables.

Logging

- Record all automation activities.
- Save logs with timestamps.
- Store logs in dedicated log files.
- Assist in debugging and monitoring.

Additional Features

- User-friendly menu-driven interface.
- Command-line execution support.
- Secure configuration management.
- Exception handling.
- Modular program structure.

Technologies Used

Technology	Purpose
Python 3	Programming Language
VS Code	Code Editor
Git	Version Control
GitHub	Project Repository
CSV	Report Generation
Gmail SMTP	Email Notifications

Python Modules Used

os

Used for directory creation, file path management, and file operations.

shutil

Used to move files between folders during organization.

csv

Used to generate structured CSV reports.

logging

Used to record application activities with timestamps.

argparse

Used to process command-line arguments.

smtplib

Used to send email notifications through SMTP.

email.message

Used to create email messages and attach reports.

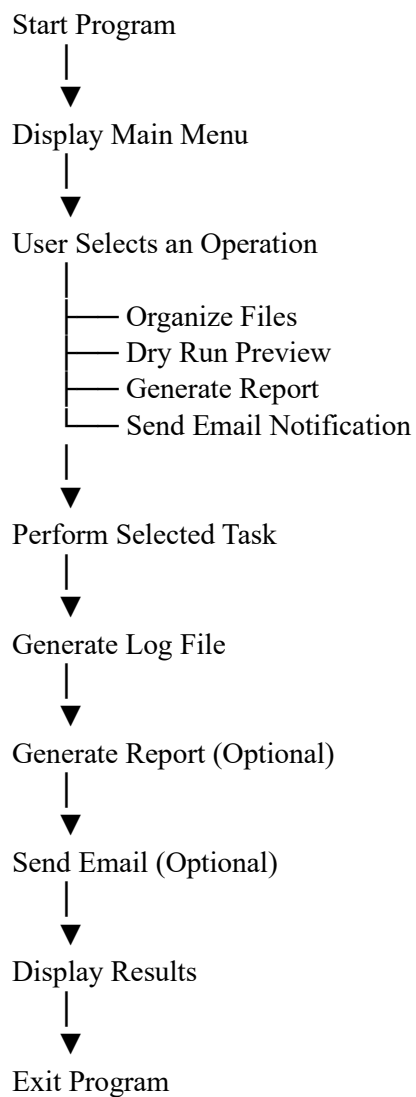
socket

Used to detect network timeout errors.

datetime

Used to generate timestamps for reports and logs.

Project Workflow



Project Structure

Task_Automation_Tool

```
|
|— task_automation_tool.py
|— sample_data/
|— reports/
|— logs/
|   — automation.log
|— README.md
```

Screenshots

```
PS C:\Users\rajes\Desktop\python\Task automation> python task_automation_tool.py

=====
TASK AUTOMATION TOOL
=====

1. Organize a folder (real run)
2. Organize a folder (dry run / preview only)
3. Generate report from last run
4. Email summary of last run
0. Exit
=====

Enter your choice: 1
Folder to organize [Enter for default: C:\Users\rajes\Desktop\python\Task automation\sample_data]: C:\Users\rajes\Desktop\Research

2026-07-07 18:22:40 | INFO | Moved '12859_2019_Article_3262.pdf' -> Documents/12859_2019_Article_3262.pdf
2026-07-07 18:22:40 | INFO | Moved '2407.16709v1.pdf' -> Documents/2407.16709v1.pdf
2026-07-07 18:22:40 | INFO | Moved '2509.01344v1.pdf' -> Documents/2509.01344v1.pdf
2026-07-07 18:22:40 | INFO | Moved '2604.28038v1.pdf' -> Documents/2604.28038v1.pdf
2026-07-07 18:22:40 | INFO | Moved '26GRI323.pdf' -> Documents/26GRI323.pdf
2026-07-07 18:22:40 | INFO | Moved 'a-deep-learning-driven-iot-systems-for-real-time-crop-health-evaluation-and-disease-prediction-IJERTV15IS060922.pdf' -> Documents/a-deep-learning-driven-iot-systems-for-real-time-crop-health-evaluation-and-disease-prediction-IJERTV15IS060922.pdf
2026-07-07 18:22:40 | INFO | Moved 'agriengineering-07-00350.pdf' -> Documents/agriengineering-07-00350.pdf
2026-07-07 18:22:40 | INFO | Moved 'agronomy-14-02208-v2.pdf' -> Documents/agronomy-14-02208-v2.pdf
2026-07-07 18:22:40 | INFO | Moved 'appls-ci-14-04322-v2.pdf' -> Documents/appls-ci-14-04322-v2.pdf
2026-07-07 18:22:40 | INFO | Moved 'A_deep_learning_model_for_predicting_risks_of_crop.pdf' -> Documents/A_deep_learning_model_for_predicting_risks_of_crop.pdf
2026-07-07 18:22:40 | INFO | Moved 'Chirita_et_al.pdf' -> Documents/Chirita_et_al.pdf
2026-07-07 18:22:40 | INFO | Moved 'Deep learning-based crop health enhancement through early disease prediction.pdf' -> Documents/Deep learning-based crop health enhancement through early disease prediction.pdf
2026-07-07 18:22:40 | INFO | Moved 'EJECE_757.pdf' -> Documents/EJECE_757.pdf
2026-07-07 18:22:40 | INFO | Moved 'electronics-14-02466.pdf' -> Documents/electronics-14-02466.pdf
2026-07-07 18:22:40 | INFO | Moved 'fpls-16-1575796.pdf' -> Documents/fpls-16-1575796.pdf
2026-07-07 18:22:40 | INFO | Moved 'horticulturae-12-00008-v2.pdf' -> Documents/horticulturae-12-00008-v2.pdf
2026-07-07 18:22:40 | INFO | Moved 'IJIRT191743_PAPER.pdf' -> Documents/IJIRT191743_PAPER.pdf
2026-07-07 18:22:40 | INFO | Moved 'Research.docx' -> Documents/Research.docx
2026-07-07 18:22:40 | INFO | Moved 'Research.docx.pdf' -> Documents/Research.docx.pdf
2026-07-07 18:22:40 | INFO | Moved 'Research.pdf' -> Documents/Research.pdf
2026-07-07 18:22:40 | INFO | Moved 's10462-024-10944-7.pdf' -> Documents/s10462-024-10944-7.pdf
2026-07-07 18:22:40 | INFO | Moved 's10661-024-12454-z.pdf' -> Documents/s10661-024-12454-z.pdf
2026-07-07 18:22:40 | INFO | Moved 's41598-024-52038-y.pdf' -> Documents/s41598-024-52038-y.pdf
2026-07-07 18:22:40 | INFO | Moved 's41598-025-26184-w.pdf' -> Documents/s41598-025-26184-w.pdf
2026-07-07 18:22:40 | INFO | Moved 's41598-025-32384-1.pdf' -> Documents/s41598-025-32384-1.pdf
2026-07-07 18:22:40 | INFO | Moved 's41598-026-49002-3.pdf' -> Documents/s41598-026-49002-3.pdf
2026-07-07 18:22:40 | INFO | Moved 'ts_43.01_18.pdf' -> Documents/ts_43.01_18.pdf
2026-07-07 18:22:40 | INFO | Moved 'Using_Deep_Learning_for_Image-Based_Plant_Disease_.pdf' -> Documents/Using_Deep_Learning_for_Image-Based_Plant_Disease_.pdf
2026-07-07 18:22:40 | INFO | Organize run complete. Summary: {'processed': 28, 'moved': 28, 'skipped': 0, 'failed': 0}

--- File Organizer Summary ---
Processed : 28
Moved : 28
Skipped : 0
Failed : 0
```

```
=====
TASK AUTOMATION TOOL
=====
1. Organize a folder (real run)
2. Organize a folder (dry run / preview only)
3. Generate report from last run
4. Email summary of last run
0. Exit
=====

Enter your choice: 2
Folder to organize [Enter for default: C:\Users\rajes\Desktop\python\Task automation\sample_data]: C:\Users\rajes\Desktop\Research
2026-07-07 18:23:34 | INFO      | Organize run complete. Summary: {'processed': 0, 'moved': 0, 'skipped': 1, 'failed': 0}

--- File Organizer (dry run) Summary ---
Processed : 0
Moved      : 0
Skipped    : 1
Failed     : 0

=====
TASK AUTOMATION TOOL
=====
1. Organize a folder (real run)
2. Organize a folder (dry run / preview only)
3. Generate report from last run
4. Email summary of last run
0. Exit
=====

Enter your choice: 3
Report format - txt or csv? [txt]: txt
2026-07-07 18:24:17 | INFO      | Text report written to C:\Users\rajes\Desktop\python\Task automation\reports\file_organizer_report_20260707_182417.txt
Report saved to: C:\Users\rajes\Desktop\python\Task automation\reports\file_organizer_report_20260707_182417.txt
```

Sample Output

```
=====
TASK AUTOMATION TOOL
=====
```

1. Organize a Folder
2. Organize a Folder (Dry Run)
3. Generate Report
4. Email Summary
0. Exit

Enter your choice: 2

----- File Organizer Summary -----

Processed Files : 6
Moved Files : 5
Skipped Files : 1
Failed Files : 0

Report generated successfully.

Error Handling

To improve reliability and security, several validation and exception handling techniques have been implemented.

The system checks for:

- Missing source folders.
- Permission denied while reading or moving files.
- Missing report files.
- Duplicate file names.
- Invalid email addresses.
- Missing email credentials.
- SMTP authentication failures.
- Network timeout errors.
- Invalid menu selections.
- File system errors.
- Missing attachments during email sending.

These validations prevent unexpected crashes and ensure smooth execution of automation tasks.

Future Enhancements

The project can be further improved by adding:

- Graphical User Interface (GUI) using Tkinter.
- Automatic folder monitoring (Watch Mode).
- PDF report generation.
- Cloud storage integration.
- Support for multiple email providers.
- File backup before organization.
- Undo operation for file movements.
- Scheduler for automatic execution.
- Web-based dashboard using Flask.

Conclusion

The **Task Automation Tool** was developed to gain practical experience in Python automation by simplifying common file management tasks. Through this project, I learned how to organize files automatically, generate reports, send email notifications, and maintain activity logs using Python's standard libraries.

Working on this project also improved my understanding of modular programming, file handling, logging, command-line programming, exception handling, and secure email integration. Features such as dry-run mode, duplicate file handling, automated reporting, and email notifications made the application more practical while enhancing my problem-solving skills.

Overall, this project strengthened my confidence in building real-world automation tools using Python and provided valuable hands-on experience that will be beneficial in future software development and automation projects.